

TEST CASE — SkyWatcher

IoT-Enabled Smart Airport Baggage Tracking System | Final Year Project (PSM), UTeM FTMK 2025/2026 | B032310853

Module 1: User Authentication & Access Control (JWT)

Test Case ID	Description	Pre-conditions	Test Steps	Expected Result	Pass / Fail
TC-001	Admin login with valid credentials	Admin account seeded; API running.	1. Open dashboard login. 2. Enter username admin / admin123 . 3. Click Login.	JWT (HS256) issued; role = admin; redirected to dashboard.	[]
TC-002	Login rejected with wrong password	Admin account exists.	1. Enter username admin and a wrong password. 2. Click Login.	401 Unauthorized; access denied; no token issued.	[]
TC-003	Ground staff sees only their checkpoint	staff_security logged in.	1. Login as staff_security . 2. Open the Bags list.	Only bags whose last checkpoint = security are shown.	[]
TC-004	Role restriction on admin-only route	Ground staff logged in.	1. As ground staff, call GET /api/stats.	403 Forbidden — insufficient permission.	[]
TC-005	Expired / invalid token blocked	Token expired or tampered.	1. Send a request with an invalid Bearer token.	401 Unauthorized; user redirected to /login.	[]

Module 2: Baggage Event Tracking

Test Case ID	Description	Pre-conditions	Test Steps	Expected Result	Pass / Fail
TC-006	Ingest a valid RFID scan event	API + broker running.	1. Publish/POST a valid event (tag_id, checkpoint, duration).	201 Created; bag upserted; event stored in DB.	[]
TC-007	Reject malformed event	API running.	1. POST an event missing tag_id and checkpoint.	400 Bad Request; "tag_id and checkpoint are required".	[]
TC-008	Duplicate scan is not double-counted	Event already stored.	1. POST the same (tag_id, checkpoint, timestamp) twice.	Duplicate ignored; only one event row persists.	[]
TC-009	Retrieve full bag history in order	Bag has several events.	1. GET /api/bags/<tag_id>/history.	All checkpoints returned in chronological order.	[]
TC-010	Public passenger self-tracking	Bag exists for flight+name.	1. GET /api/track?flight_id=MH370&passenger=Ahmad.	Bag + event timeline returned; no login required.	[]

Module 3: Real-Time Anomaly Detection (Isolation Forest + Rules)

Test Case ID	Description	Pre-conditions	Test Steps	Expected Result	Pass / Fail
TC-011	Normal bag → no anomaly	History: check_in → security.	1. Process a security event with duration 4 min.	No anomaly raised (returns None).	[]
TC-012	STALL detection (> 20 min)	Threshold = 20 min.	1. Process a check_in event with duration 25 min.	Anomaly type = STALL raised and stored.	[]
TC-013	STALL boundary (exactly 20 min)	Threshold = 20 min.	1. Process an event with duration exactly 20.0 min.	STALL fires (threshold is inclusive).	[]
TC-014	WRONG_ROUTE (skips a step)	Previous checkpoint = check_in.	1. Process a sorting event (skips security).	Anomaly type = WRONG_ROUTE raised.	[]
TC-015	SECURITY_BYPASS (no security scan)	History has no security event.	1. Bag reaches loading without a security scan.	Anomaly type = SECURITY_BYPASS raised.	[]
TC-016	No false alarm on valid sequence	History: check_in → security → sorting.	1. Process the sorting event normally.	No SECURITY_BYPASS; no anomaly.	[]
TC-017	Confidence score is bounded	Any detected anomaly.	1. Inspect the score of a raised anomaly.	Score is within [0.0, 1.0]; alert persisted once.	[]

Module 4: Alerts & Live Dashboard

Test Case ID	Description	Pre-conditions	Test Steps	Expected Result	Pass / Fail
TC-018	Anomaly appears as an alert	An anomaly was detected.	1. GET /api/alerts.	Alert listed with type, checkpoint and unresolved count.	[]
TC-019	Resolve an alert	Unresolved alert exists.	1. PATCH /api/alerts/<id>/resolve.	Alert marked resolved with resolved_at timestamp.	[]
TC-020	Staff cannot resolve other checkpoints	Alert belongs to another checkpoint.	1. Ground staff attempts to resolve it.	403 Forbidden — only own-checkpoint alerts allowed.	[]
TC-021	Live dashboard auto-refresh	Dashboard open.	1. Create a new event and wait one poll cycle.	New data appears within the 3-second poll interval.	[]

Module 5: Time Requirement Test (Real-Time Performance)

SkyWatcher is a real-time system: each scan must be ingested, analysed and surfaced fast enough for ground staff to act while the bag is still in the hall. The database layer is mocked so each figure reflects the system's own processing time, not network latency. Automated with pytest (500–1000 iterations, warmed up).

Test Case ID	Description	Method	Requirement (Threshold)	Measured Result	Pass / Fail
TC-T01	Mean anomaly-detection latency per event	500 detections, mean of per-call time	≤ 50 ms	≈ 2.1 ms	[✓]
TC-T02	95th-percentile detection latency	500 detections, p95 (nearest rank)	≤ 100 ms	≈ 2.2 ms	[✓]
TC-T03	Worst-case detection latency	500 detections, maximum	≤ 250 ms	≈ 7 ms	[✓]

Test Case ID	Description	Method	Requirement (Threshold)	Measured Result	Pass / Fail
TC-T04	Machine-learning inference only	500 IsolationForest.predict() calls	≤ 20 ms	≈ 2.1 ms	[✓]
TC-T05	Sustained processing throughput	1000 sequential detections	≥ 100 events / s	≈ 470 events/s	[✓]
TC-T06	Cold first-scan latency after start-up	First timed detection call	≤ 500 ms	≈ 2.1 ms	[✓]
TC-T07	End-to-end POST /api/events (live stack)	20 live HTTP requests, mean	≤ 1500 ms	measured at demo	[]
TC-T08	GET /api/health round-trip (live stack)	20 live HTTP requests, mean	≤ 500 ms	measured at demo	[]

Conclusion: detection completes in ≈2 ms — about 25× faster than the 50 ms budget — and sustains ≈470 events per second on a single thread. As this is far below the 3-second dashboard refresh interval, anomalies are effectively surfaced on the next poll, satisfying the real-time requirement. (Millisecond values vary slightly per machine; the test asserts against the threshold, not the exact number.)

Legend: [] = to be verified during demonstration [✓] = automated test executed and passed. Automated cases: run `cd backend && python -m pytest`.